MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)
Mini-Project Title: **VKU Field Survey — Offline-First Facility Inspection App**
Student Name: Nguyễn Thái Sơn
Submission Date: 03/09/2026

---

1. GENERAL INFORMATION & DELIVERABLE LINKS

- **Team Members:** Nguyễn Thái Sơn — Student ID: **23IT.EB086** — Role: *Fullstack Architecture & PWA / Capacitor Integration* — Contribution: **100%**
- **Live Demo URL:** https://vku-survey.pages.dev
- **GitHub Repository:** https://github.com/nguyenthaison2408/vku-survey
- 📹 **Video Demo:**

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	**Responsive Mobile Viewport & PWA Standalone**	✅ **Complete**	100% responsive with Tailwind CSS, VKU theme palette. PWA manifest (display: standalone, custom icons, theme-color) installable on Android/iOS.
2	**Local Offline Persistence (IndexedDB)**	✅ **Complete**	Uses idb (IndexedDB wrapper) with 3 object stores (surveys, drafts, settings). Auto-saves drafts real-time across 5 form steps without data loss.
3	**Automatic Background Sync & Offline Queue**	✅ **Complete**	Payloads submitted offline transition to PENDING_SYNC. Auto-flushed and synced via Service Worker sync event + Capacitor Network + window.ononline.
4	**Hardware Device Integration (Camera API)**	✅ **Complete**	Cross-platform camera capture using @capacitor/camera for native Android with HTML5 file input fallback for standard web browsers.
5	**Multi-step Inspection Workflow & Dynamic Rating**	✅ **Complete**	5-step guided wizard (Location → Category → 5-Star/Emoji Rating → Notes → Photo Preview) with real-time field validation and feedback toasts.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

3.1. Project Directory Tree

```
src/ ├── main.ts              # Application bootstrap & PWA Service Worker
registration ├── sw.ts                # Service Worker: Cache-First strategy &
Background Sync handler ├── db.ts               # IndexedDB database management (CRUD
for surveys, drafts, settings) ├── sync.ts             # Offline synchronization
engine & API queue dispatcher ├── camera.ts           # Capacitor Camera integration
with web input fallback ├── network.ts           # Hybrid network monitor (Capacitor
Network + Navigator API) ├── router.ts            # Lightweight Hash-based SPA Router
(#/, #/survey, #/history) ├── style.css            # Tailwind directives, animations &
responsive mobile styling ├── pages/ │   ├── home.ts          # Metrics dashboard
(Total, Pending, Synced count) & quick actions │   ├── survey.ts        # 5-step
inspection form wizard with real-time draft persistence │   └── history.ts       #
Filterable survey log with sync status badges └── components/     ├── toast.ts
# Non-blocking animated alert notifications     └── star-rating.ts   # Interactive 2-
way synchronized star and emoji rating component
```

3.2. State Management & Offline-First Flow

- **Local Persistence First:** All user actions write immediately to IndexedDB (drafts store). Submitting an inspection updates the local record status to PENDING_SYNC before attempting any network transport.
- **Network-Aware Sync Dispatcher:** A reactive network listener (network.ts) notifies the UI via status bars. When connectivity returns: Offline Submission → IndexedDB: PENDING_SYNC → Online Trigger → Sequential POST API → SYNCED.
- **Exception Handling Strategy:** Network errors or server downtimes increment syncAttempts on each survey record without blocking UI execution. Errors trigger animated toast notifications (toast.ts) while keeping payloads safe in storage.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

Figure 1: Dashboard & Online/Offline Monitor

Description: The home screen displays statistical counters (completed, pending synchronization, completed), a real-time network connection status bar, and a quick test start button.

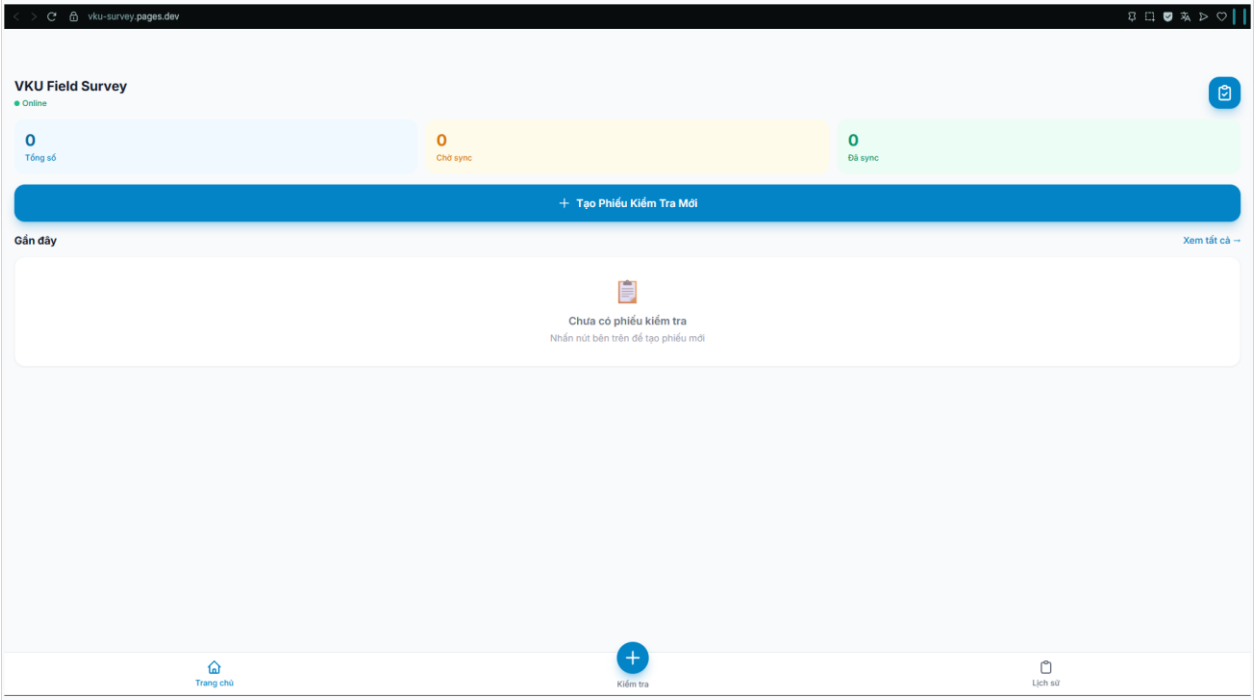


Figure 2: Multi-step Inspection Form with Interactive Rating

Description: The Step 3 interface assesses device status using a 5-star rating system and five emotion buttons (Very Poor, Poor, Average, Good, Excellent) with two-way status synchronization.

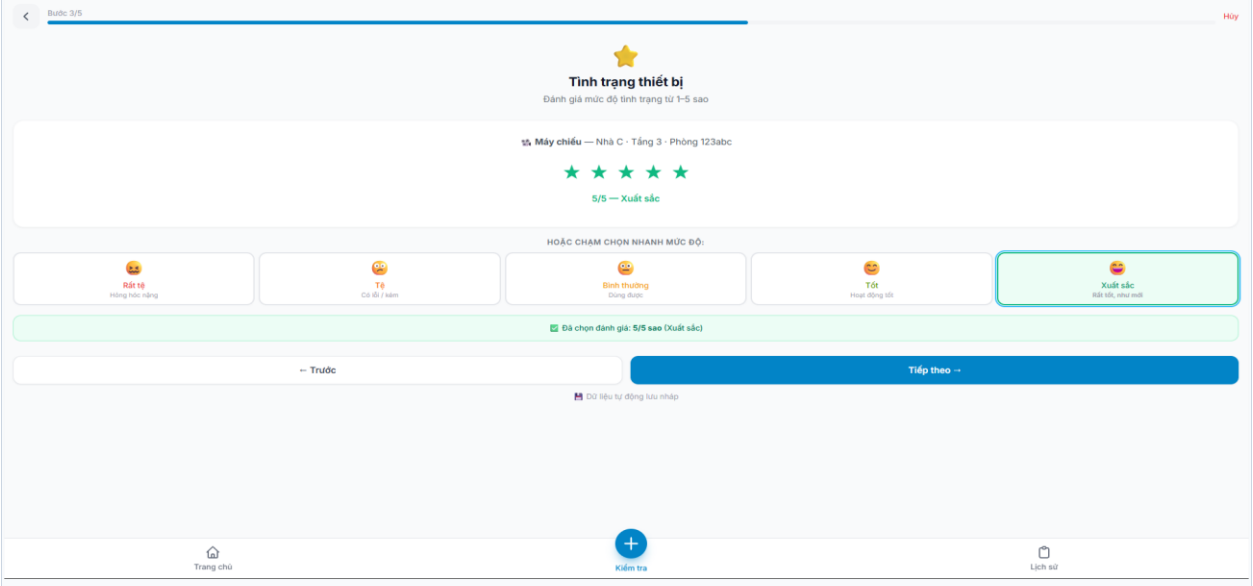


Figure 3: Hardware Camera Capture & Evidence Preview

Description: Opens the device camera to capture an image, compresses the image to Base64, and displays a preview with options to retake or delete the photo.

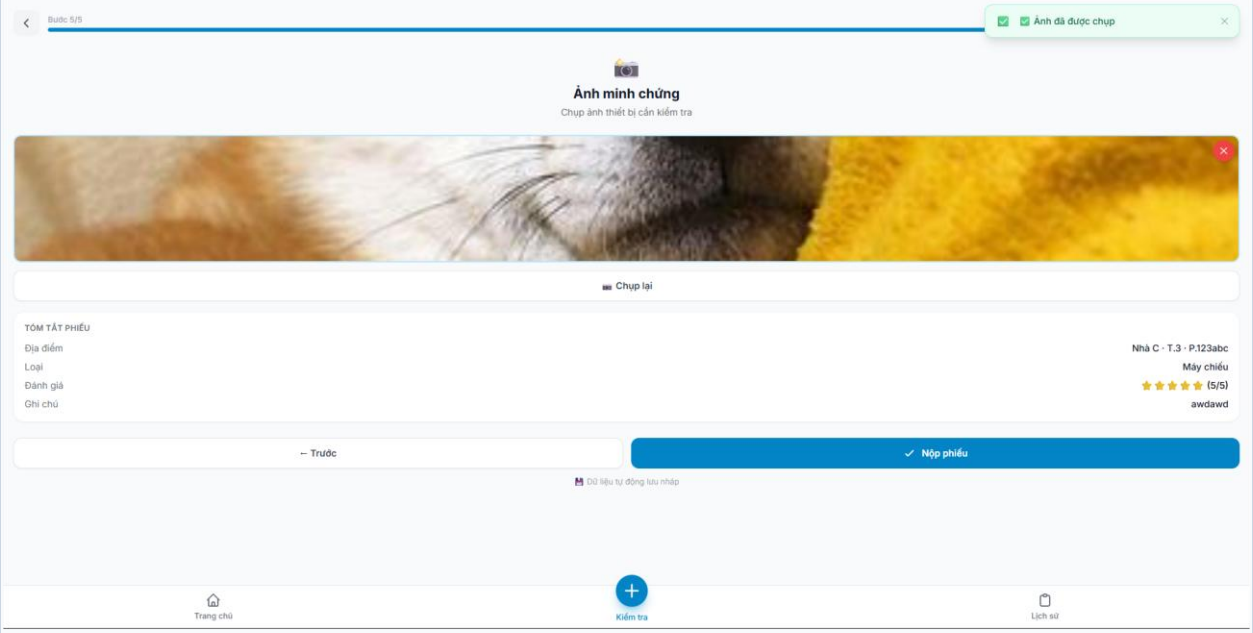
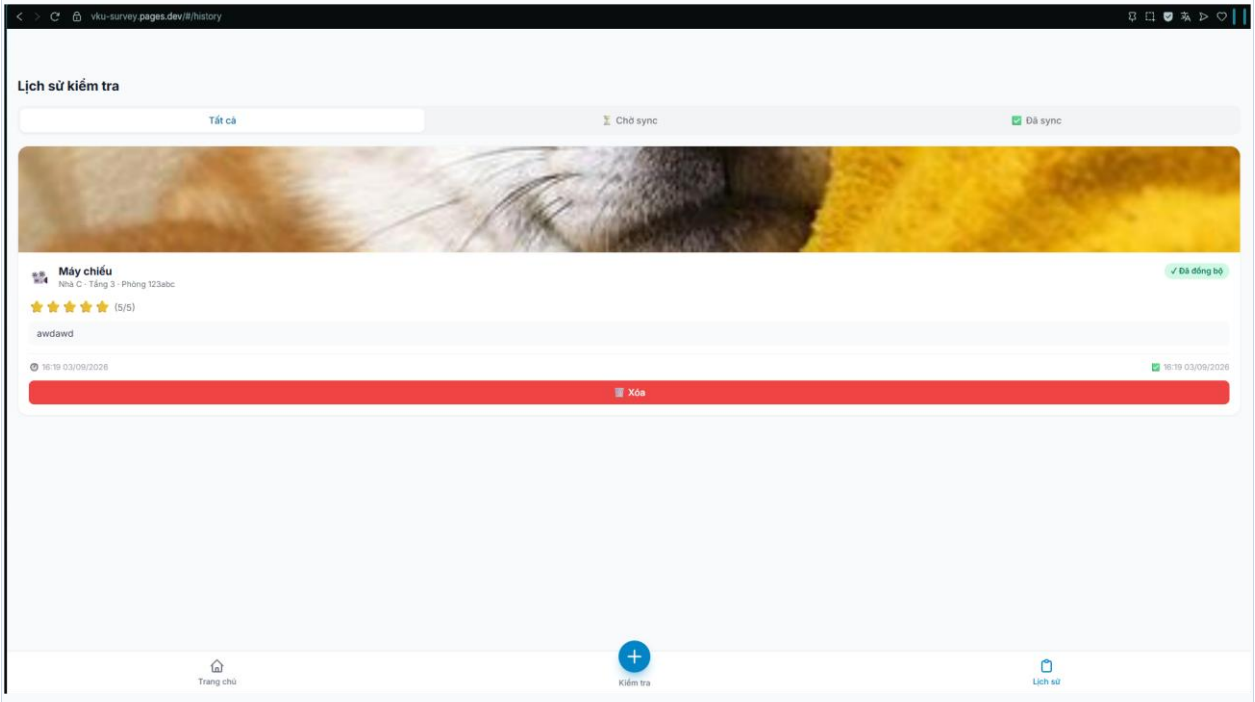


Figure 4: Historical Audit Trail & PWA Installation

Description: Inspection history list filtered by status (All, Pending, Synced) and notification for installing the PWA in standalone mode on mobile devices.



5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1: DOM Desynchronization & Event Dropping in Dynamic Rating Component

- **Issue:** Initially, hovering over or touching the rating stars destroyed and recreated the DOM subtree (container.innerHTML = ...). This caused rapid DOM recreation loops, tearing down buttons mid-click and dropping tap/click events, leading to a false validation failure error ("Vui lòng đánh giá tình trạng") when moving to the next step.
- **Resolution:** Refactored star-rating.ts to establish a static, persistent DOM structure created once upon mounting. Replaced innerHTML destruction with direct element class/attribute mutations. In addition, transformed the 5 static emoji status indicators into interactive buttons synchronized two-way with the stars, ensuring reliable input registration across both desktop mice and mobile touchscreens.

Challenge 2: Cross-Browser Offline Background Sync Inconsistencies

- **Issue:** The standard W3C Background Sync API (registration.sync.register) is natively supported on Chromium-based Android browsers but restricted or missing on iOS WebKit (Safari). Relying solely on Service Worker sync events led to stale pending queues on certain mobile devices.
- **Resolution:** Engineered a resilient, multi-tiered synchronization fallback in sync.ts. The engine attempts native Service Worker Background Sync first; if unavailable, it seamlessly falls back to combining @capacitor/network status change events and window.addEventListener('online') hooks to instantly flush the offline queue upon network restoration.